UNIVERSITÉ INTERNATIONALE DE RABAT

Génie Informatique — Big Data & Intelligence Artificielle

PROJET DE FIN D'ÉTUDES

EquiVision AI

Application web expérimentale de gestion et d'analyse équine

Auteur : El Hani El Mehdi

Filière : Génie Informatique — Spécialité Big Data & AI

Année universitaire : 2025 – 2026

Encadrant académique : [Nom de l'encadrant à compléter]

Remerciements

Je tiens à exprimer ma sincère gratitude à l'ensemble du corps enseignant de l'Université Internationale de Rabat, ainsi qu'à mon encadrant académique, pour l'accompagnement, les conseils et le suivi apportés durant la réalisation de ce Projet de Fin d'Études. Je remercie également les personnes ayant contribué aux essais et à l'amélioration du projet, ainsi que ma famille et mes proches pour leur soutien.

Résumé

Ce rapport présente EquiVision AI, une plateforme web expérimentale conçue pour centraliser la gestion des chevaux, des cavaliers, des entraînements, des compétitions, du calendrier et du suivi médical. Développée avec React et TypeScript côté frontend, FastAPI côté backend et SQLite pour la persistance, la solution intègre également un module de Machine Learning basé sur Random Forest.

Le module IA exploite cinq variables — âge, fréquence cardiaque, température, volume d'entraînement et historique de blessures — afin de produire une classification expérimentale en trois catégories : Healthy, Warning et Risk. L'objectif est de démontrer l'intégration d'un modèle de classification dans une application web, et non de fournir un diagnostic vétérinaire.

Le projet est volontairement présenté comme un prototype académique. Les performances et les limites du modèle sont analysées avec prudence, notamment l'accuracy expérimentale de 50 % observée lors d'un test antérieur.

Abstract

This report presents EquiVision AI, an experimental web platform designed to centralize horse, rider, training, competition, calendar and medical information. The solution combines a React/TypeScript frontend, a FastAPI backend, SQLite persistence and a Random Forest machine-learning component.

The AI component uses age, heart rate, temperature, training hours and injury history to produce an experimental three-class classification: Healthy, Warning and Risk. It is presented as a proof of concept and decision-support component, not as a veterinary diagnostic system.

Mots-clés

Intelligence artificielle — Machine Learning — Random Forest — gestion équine — FastAPI — React — classification — aide à la décision

Sommaire

Le sommaire doit être généré automatiquement dans Word après insertion des titres. Utiliser Références → Table des matières.

Liste des figures

Architecture générale

Interface de connexion

Tableau de bord

Liste des chevaux

Module Cavaliers

Calendrier

Entraînements

Compétitions

Rapports PDF

Module Santé & IA

Liste des tableaux

Variables utilisées par le modèle

Comparaison des algorithmes

Résultats d'évaluation

Cas de test globaux

Liste des acronymes

Acronyme	Signification
PFE	Projet de Fin d'Études
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
IA	Intelligence Artificielle
ML	Machine Learning
CRUD	Create, Read, Update, Delete
ORM	Object-Relational Mapping
UI/UX	User Interface / User Experience
PDF	Portable Document Format

1. Introduction générale

1.1 Contexte

Le domaine équestre génère quotidiennement des informations liées aux chevaux, aux cavaliers, aux entraînements, aux compétitions et au suivi médical. Lorsque ces informations sont réparties entre plusieurs supports, leur consultation et leur mise à jour deviennent moins efficaces.

Parallèlement, les technologies web et l'intelligence artificielle permettent aujourd'hui de construire des applications capables de centraliser les données et d'apporter des analyses automatisées. Cette évolution constitue une opportunité pour expérimenter des outils numériques adaptés au suivi d'une activité équestre.

1.2 Pourquoi EquiVision AI a-t-il été créé ?

EquiVision AI a été créé dans le but de proposer, dans un contexte académique, une application unique permettant de regrouper les principales informations d'une activité équestre et d'explorer l'apport du Machine Learning au suivi de la santé des chevaux. Le projet cherche ainsi à relier développement web, gestion de données et intelligence artificielle autour d'un cas d'usage concret.

1.3 Problématique

Comment concevoir une application web expérimentale capable de centraliser les informations essentielles d'une activité équestre, tout en intégrant un module de Machine Learning permettant de produire une classification indicative de l'état de santé d'un cheval à partir de données physiologiques et d'entraînement ?

1.4 Objectifs

- Centraliser les données relatives aux chevaux, cavaliers, entraînements, compétitions, calendrier et dossiers médicaux.
- Développer une interface web moderne et structurée.
- Mettre en place un backend REST sécurisé par authentification JWT.
- Intégrer un modèle Random Forest dans une API FastAPI.
- Permettre une classification expérimentale en trois catégories : Healthy, Warning et Risk.
- Générer des rapports PDF.
- Évaluer les limites du prototype et proposer des évolutions réalistes.

1.5 Méthodologie

Le projet a été réalisé selon une démarche itérative : identification des besoins, conception de l'architecture, développement progressif du frontend et du backend, intégration de la base de données, développement du modèle IA, intégration de l'API de prédiction et vérification des fonctionnalités.

1.6 Organisation du rapport

Le chapitre 2 présente l'étude des besoins et l'analyse du domaine. Le chapitre 3 décrit la conception et l'architecture. Le chapitre 4 présente la réalisation des modules. Le chapitre 5 est consacré à l'intelligence artificielle. Le chapitre 6 présente les tests. Le chapitre 7 discute les limites et les perspectives, avant la conclusion générale.

2. Étude et analyse des besoins

2.1 Présentation du domaine

Une activité équestre nécessite le suivi de données administratives, sportives et médicales. Le cheval constitue l'élément central, mais son suivi implique également les cavaliers, les séances d'entraînement, les compétitions et les événements planifiés.

2.2 Constat et besoin

Le besoin identifié est de disposer d'un espace numérique unique permettant de consulter et de mettre à jour ces informations sans multiplier les supports. L'intérêt d'EquiVision AI réside également dans la possibilité d'exploiter les données enregistrées pour expérimenter une analyse automatisée.

2.3 Besoins fonctionnels

- Authentification et gestion de session.
- Gestion CRUD des chevaux.
- Gestion des cavaliers.
- Gestion des dossiers médicaux.
- Suivi des entraînements.
- Gestion des compétitions.
- Calendrier des activités.
- Génération de rapports PDF.
- Analyse expérimentale par IA.
- Tableau de bord de synthèse.

2.4 Besoins non fonctionnels

- Ergonomie et lisibilité de l'interface.
- Séparation frontend/backend.
- Validation des données côté serveur.
- Authentification par JWT.
- Facilité de déploiement local avec SQLite.
- Maintenabilité et évolutivité de l'architecture.

2.5 Acteurs et cas d'utilisation

Acteur	Rôle
Utilisateur	Consulter et gérer les données de l'écurie et lancer les analyses.
Module IA	Recevoir les variables d'entrée et produire une classification expérimentale.

Les cas d'utilisation principaux sont : s'authentifier, gérer un cheval, consulter son profil, gérer un dossier médical, planifier un entraînement, gérer une compétition, consulter le calendrier, lancer une analyse IA et générer un rapport.

3. Conception et architecture

3.1 Architecture globale

EquiVision AI adopte une architecture en couches séparant l'interface utilisateur, la logique applicative et la persistance. Le modèle IA est intégré au backend afin que le frontend puisse l'utiliser via une API.

3.2 Frontend

Le frontend repose sur React et TypeScript, avec Vite pour le développement, Tailwind CSS pour l'interface, React Router pour la navigation et Axios pour les échanges avec l'API.

3.3 Backend

Le backend est construit avec Python et FastAPI. SQLAlchemy assure l'accès ORM à SQLite et Pydantic participe à la validation des données. Uvicorn assure l'exécution du serveur.

3.4 Architecture IA

Le modèle Random Forest est chargé côté backend et utilisé lors des requêtes de prédiction. Cette organisation évite de placer la logique du modèle directement dans l'interface utilisateur.

3.5 Base de données

SQLite est utilisée comme solution de persistance adaptée au contexte du prototype. SQLAlchemy permet de manipuler les entités de manière structurée.

3.6 API REST

Les échanges entre le frontend et le backend utilisent des requêtes HTTP et des données JSON. Deux routes IA importantes sont prévues : GET /ai/status pour l'état du service et POST /ai/predict pour la classification.

3.7 Sécurité

L'authentification utilise JWT et le hachage des mots de passe repose sur Passlib/bcrypt. Le frontend gère notamment les réponses 401 afin de traiter les sessions invalides.

3.8 Conception UML

Les diagrammes UML — cas d'utilisation, classes et séquences — sont à insérer ici s'ils ont été réalisés. Ils doivent représenter le fonctionnement réel de la version finale.

4. Développement de la plateforme

4.1 Environnement technique

Couche	Technologies
Frontend	React, TypeScript, Vite, Tailwind CSS, React Router, Axios
Backend	Python, FastAPI, Uvicorn, Pydantic, SQLAlchemy
Base de données	SQLite
IA	Scikit-learn, NumPy, Pandas, Joblib
Sécurité	JWT, Passlib, bcrypt
Rapports	jsPDF

4.2 Authentification

L'interface de connexion permet à l'utilisateur de fournir son adresse e-mail et son mot de passe. Après authentification, le jeton JWT est utilisé pour sécuriser les requêtes vers l'API.

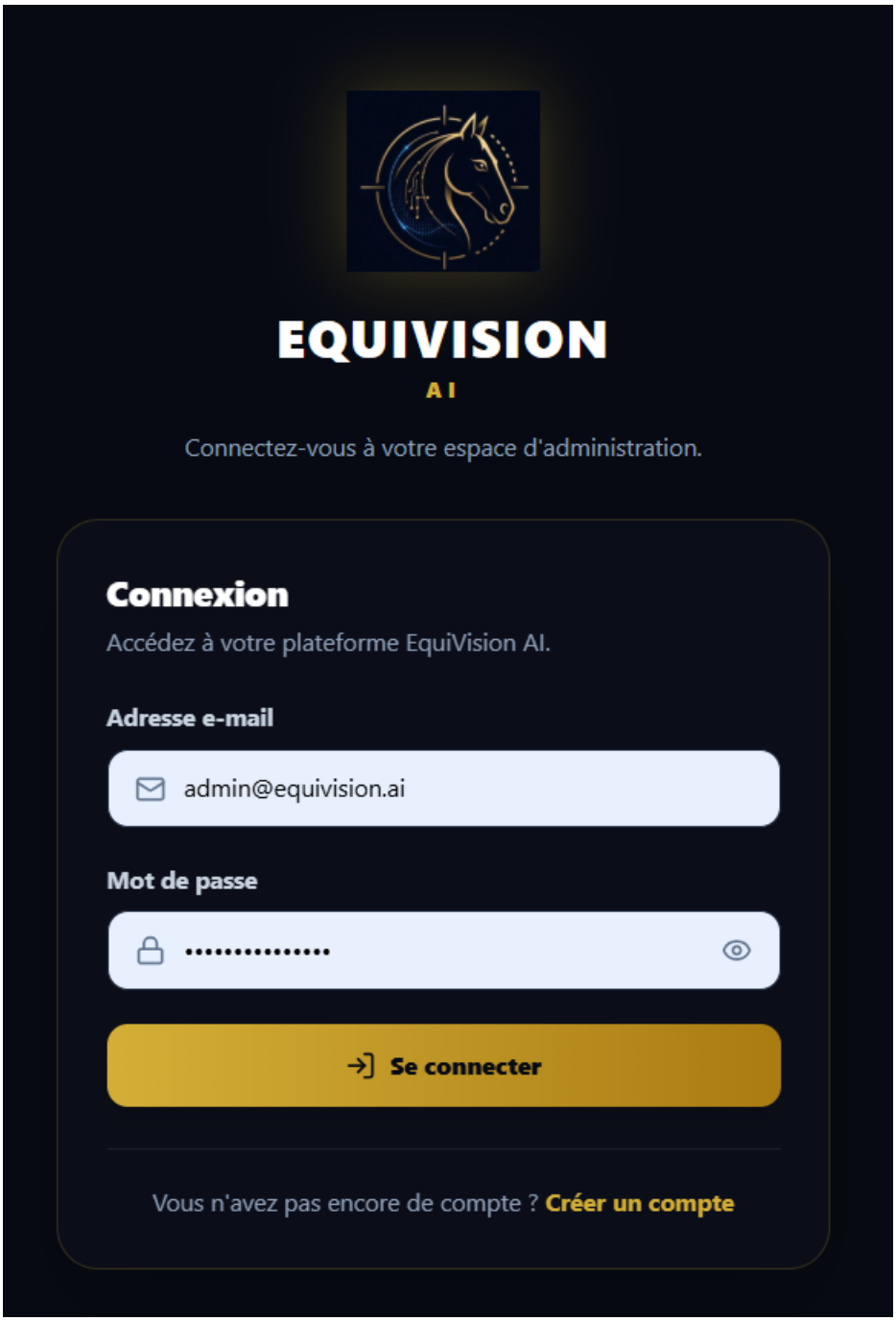


Figure 1 — Interface de connexion à la plateforme EquiVision AI

4.3 Tableau de bord

Le tableau de bord fournit une vue synthétique de l’activité et donne accès aux principaux modules de la plateforme.

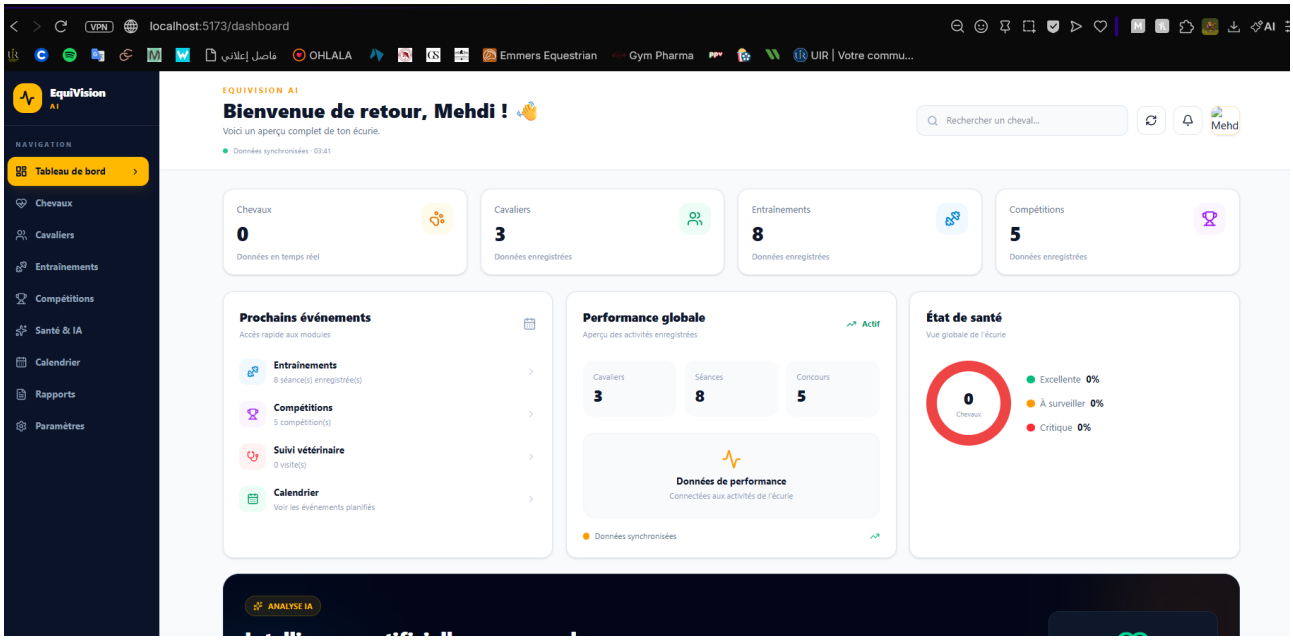


Figure 2 — Tableau de bord de la plateforme EquiVision AI

4.4 Gestion des chevaux

Le module Chevaux permet de créer, consulter, modifier et supprimer les fiches des chevaux. Les informations affichées comprennent notamment l'identité et plusieurs caractéristiques générales.

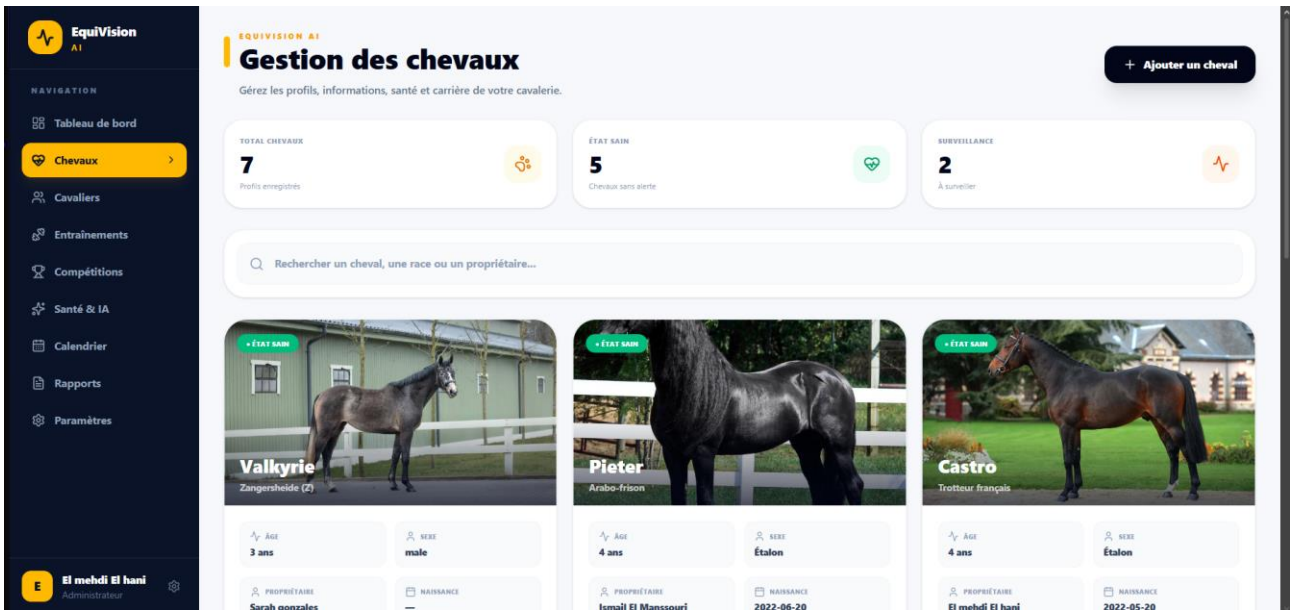


Figure 3 — Liste des chevaux enregistrés

4.5 Cavaliers

Le module Cavaliers regroupe les informations relatives aux cavaliers et présente des indicateurs liés à leur activité dans l'application.

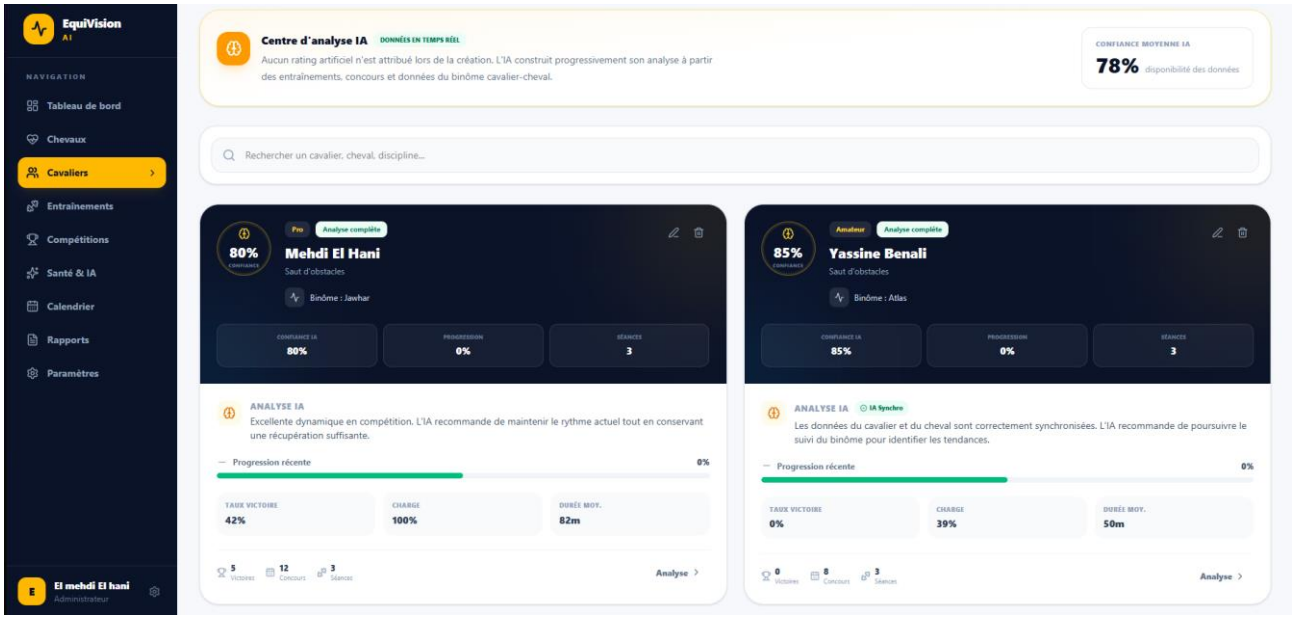


Figure 4 — Module de gestion des cavaliers

4.6 Dossier médical

Le dossier médical centralise les informations de suivi telles que la date de visite, le vétérinaire, la température, la fréquence cardiaque, le poids, le diagnostic, le traitement, la vaccination, la condition et les notes.

4.7 Calendrier

Le calendrier permet de visualiser les activités planifiées et de regrouper les événements de l'écurie dans une même interface.

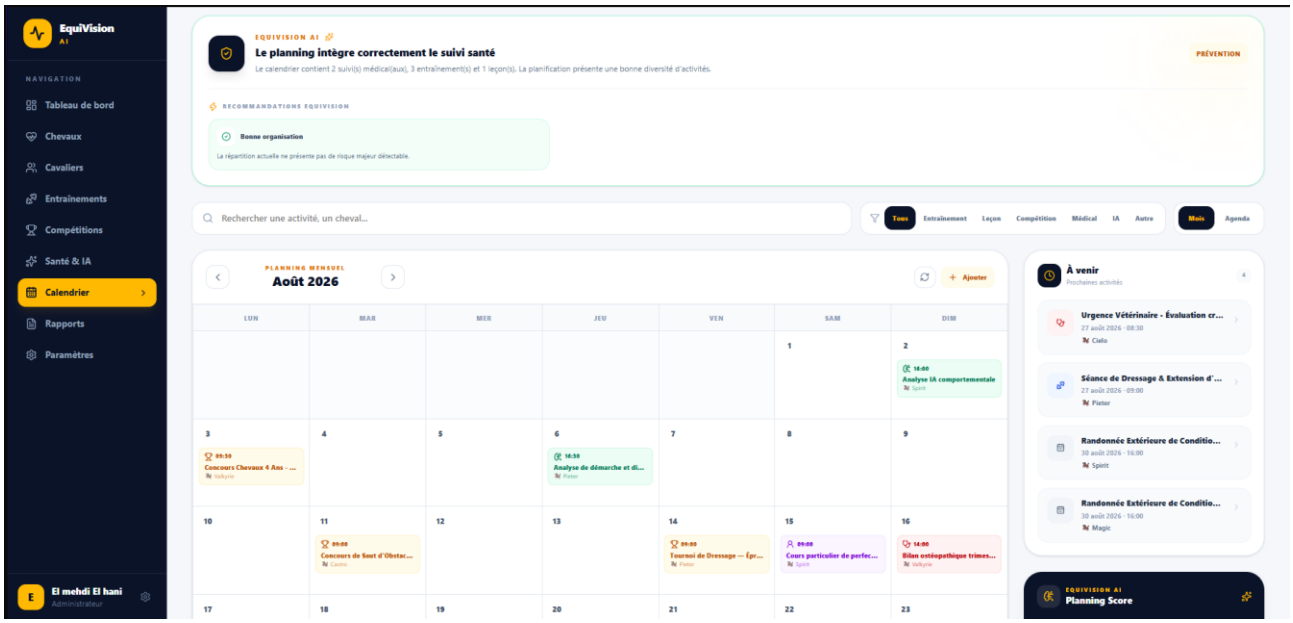


Figure 5 — Vue calendrier des activités

4.8 Entraînements

Le module Entraînements permet de suivre les séances et leurs principales caractéristiques : cheval, cavalier, date, discipline, durée et intensité.

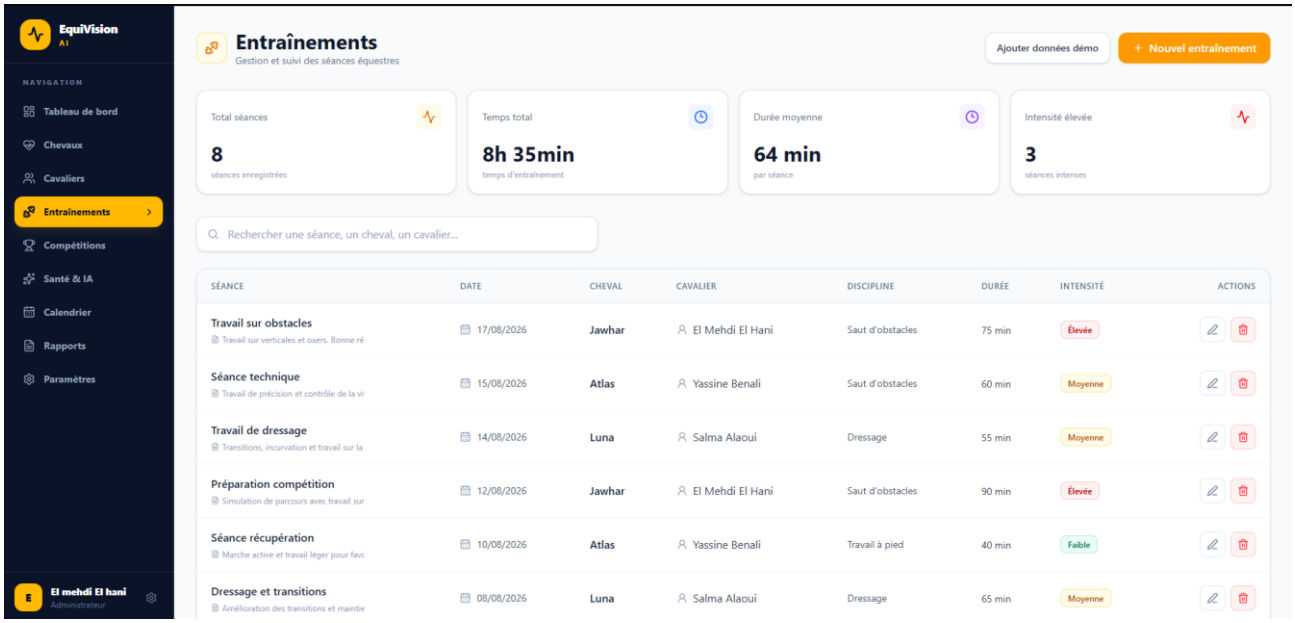


Figure 6 — Module de suivi des entraînements

4.9 Compétitions

Le module Compétitions permet d'organiser les événements sportifs et de suivre les informations associées aux engagements.

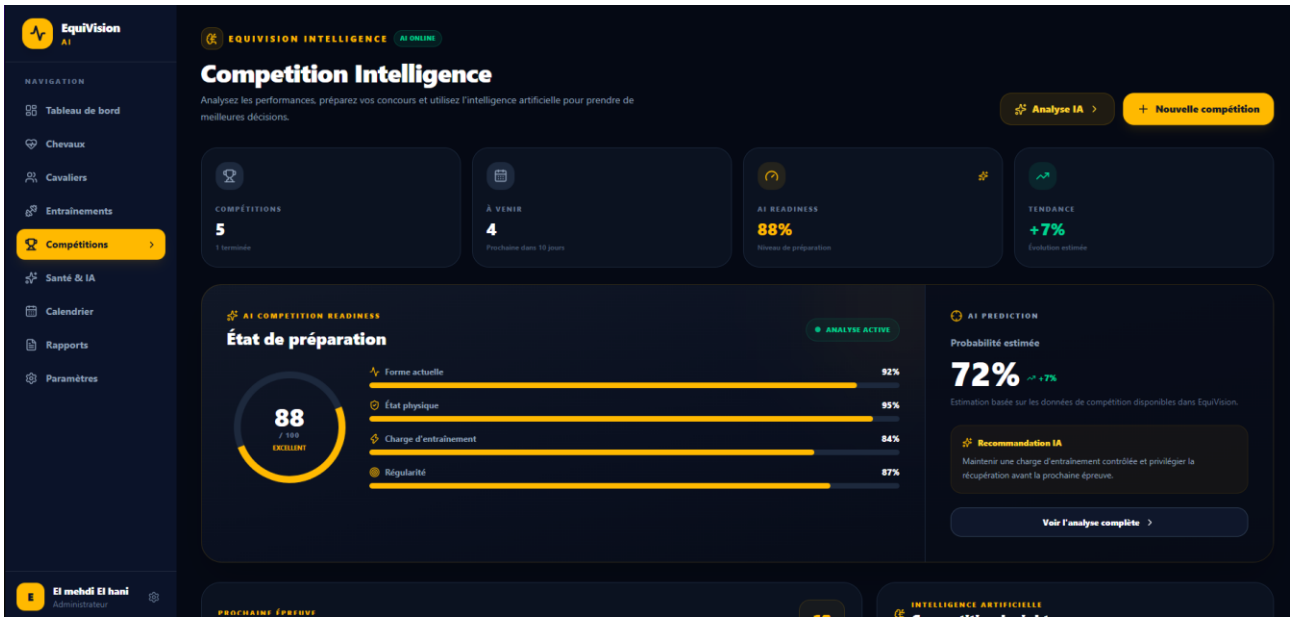


Figure 7 — Module de gestion des compétitions

4.10 Rapports PDF

La génération de rapports permet de produire une synthèse des informations disponibles dans l'application au format PDF grâce à jsPDF.

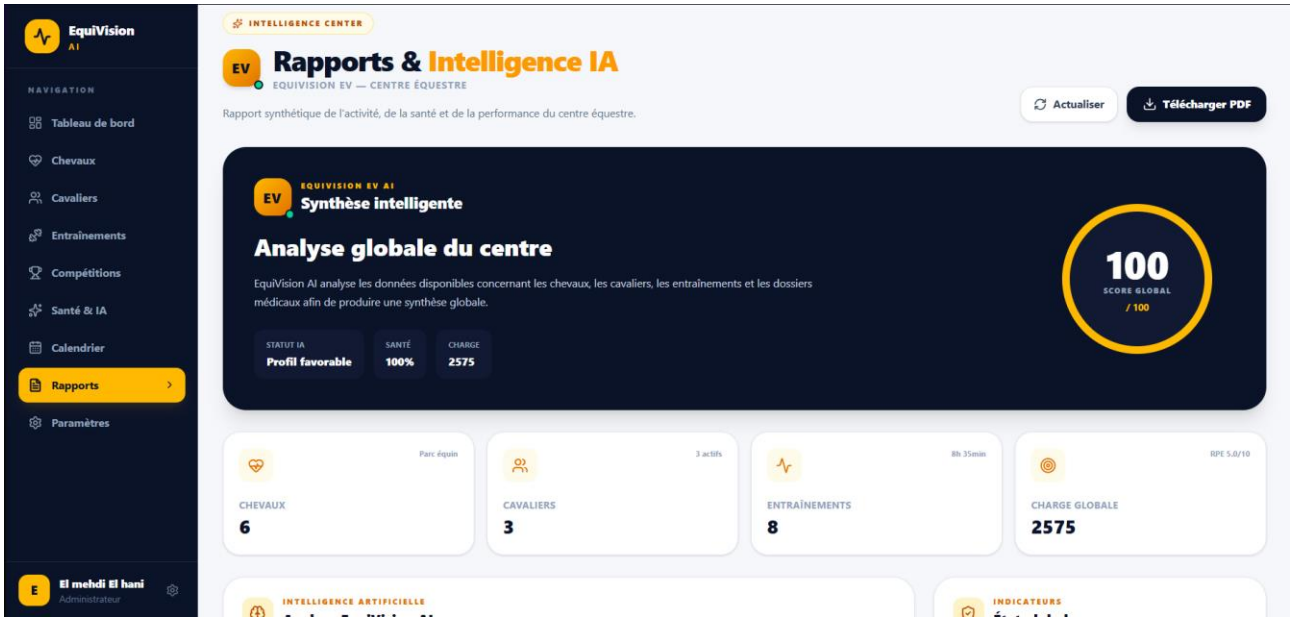


Figure 8 — Interface de génération de rapports

5. Intelligence artificielle

5.1 Objectif du module IA

L'intelligence artificielle constitue l'élément différenciant du projet. L'objectif est d'expérimenter un modèle de classification capable de produire une indication parmi trois catégories : Healthy, Warning et Risk. Cette indication doit être considérée comme une aide à la surveillance et non comme un diagnostic vétérinaire.

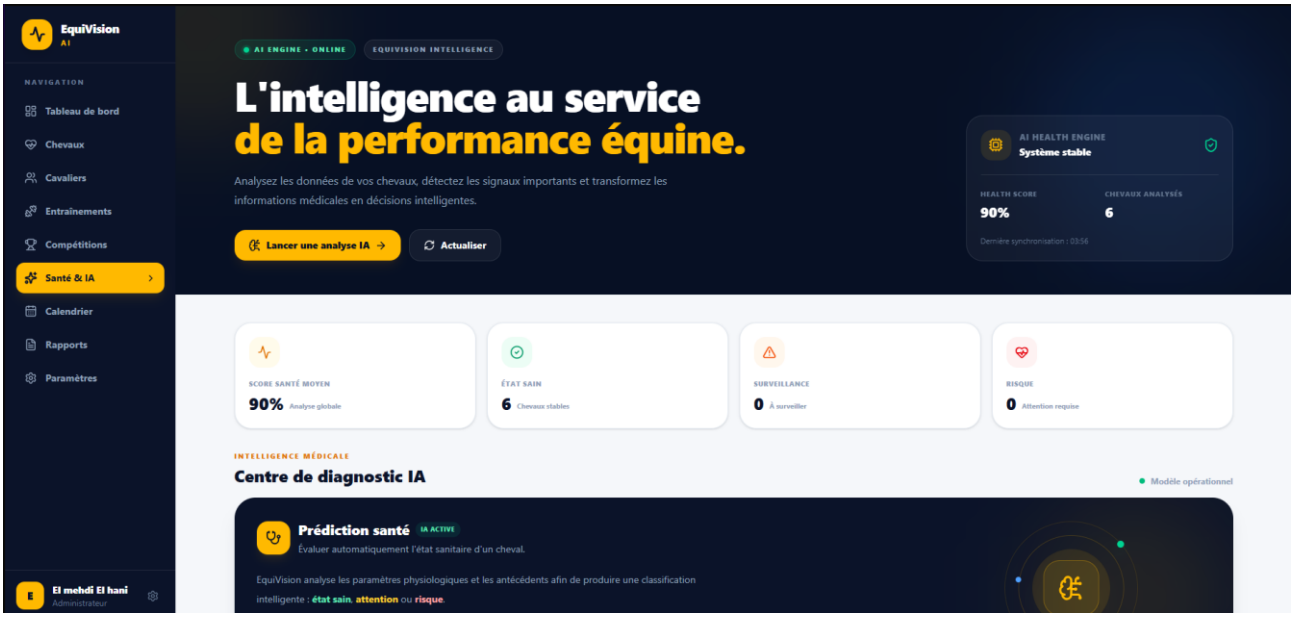


Figure 9 — Module Santé & IA

5.2 Données et variables

Variable	Description
age	Âge du cheval
heart_rate	Fréquence cardiaque
temperature	Température corporelle
training_hours	Volume d'entraînement
injury_history	Historique de blessures

5.3 Prétraitement

Les étapes exactes de préparation des données doivent être décrites à partir du script d'entraînement final : gestion des valeurs manquantes, séparation des données, éventuelle normalisation et encodage. Aucune étape non vérifiée ne doit être présentée comme réalisée.

5.4 Choix de Random Forest

Random Forest a été retenu pour sa capacité à combiner plusieurs arbres de décision et à modéliser des relations non linéaires. Il constitue un choix adapté à une expérimentation de classification sur des variables hétérogènes.

5.5 Comparaison

Algorithme	Avantages	Limites
Régression logistique	Simple et interprétable	Relations linéaires
Arbre de décision	Facile à comprendre	Risque de surapprentissage
SVM	Puissant sur certains jeux de données	Réglage et interprétation plus complexes
KNN	Simple à mettre en œuvre	Sensible à l'échelle
Random Forest	Robuste, relations complexes	Nécessite des données suffisantes

5.6 Entraînement et évaluation

Un résultat expérimental précédemment observé indique une accuracy de 0,5, soit 50 %. Ce résultat ne permet pas de conclure à une fiabilité suffisante pour un usage réel. Les autres métriques doivent être ajoutées uniquement lorsqu'elles ont été effectivement calculées.

Métrique	Résultat
Accuracy	0,50 (50 %) — résultat expérimental observé
Precision	Non mesurée
Recall	Non mesurée
F1-score	Non mesuré
Matrice de confusion	Non produite dans la version documentée

5.7 API IA

GET /ai/status permet de vérifier l'état du service et le chargement du modèle. POST /ai/predict reçoit les cinq variables d'entrée et retourne la classe prédite selon l'implémentation du backend.

5.8 Intégration dans le frontend

L'interface Santé & IA permet de sélectionner un cheval et de lancer une analyse. Elle présente l'état du service IA et les informations de classification disponibles.

5.9 Limites

- Performance expérimentale de 50 %, insuffisante pour une utilisation réelle.
- Jeu de données à documenter précisément.
- Absence de validation vétérinaire.
- Risque de biais et de mauvaise généralisation.
- Modèle destiné à l'expérimentation et non au diagnostic.

6. Tests et validation

6.1 Stratégie de test

La validation couvre l'authentification, les opérations principales, les API, la navigation, le module IA et la génération des PDF. Les statuts doivent être renseignés à partir de tests réellement exécutés.

6.2 Cas de test

ID	Module	Test	Résultat attendu	Statut
T01	Authentification	Login valide	Accès au tableau de bord	À tester
T02	Authentification	Mot de passe erroné	Message d'erreur	À tester
T03	Chevaux	Création	Cheval ajouté	À tester
T04	Chevaux	Modification	Données mises à jour	À tester
T05	Chevaux	Suppression	Cheval supprimé	À tester
T06	Médical	Création dossier	Dossier créé	À tester
T07	IA	GET /ai/status	Modèle chargé	À tester
T08	IA	POST /ai/predict	Classification retournée	À tester
T09	IA	Entrée invalide	Erreur gérée	À tester
T10	Navigation	Routes principales	Pages accessibles	À tester
T11	PDF	Génération	PDF créé	À tester
T12	PDF	Téléchargement	Fichier exploitable	À tester

6.3 Résultats

Les résultats définitifs doivent être complétés après la campagne de tests finale. Cette règle évite d'inventer des performances ou des validations qui n'ont pas été observées.

7. Discussion, limites et perspectives

7.1 Le projet répond-il au problème initial ?

EquiVision AI répond à l'objectif de centralisation en regroupant plusieurs fonctions de gestion dans une même application. Le module IA apporte une première expérimentation de classification, mais sa performance actuelle limite sa portée.

7.2 Quelle est la valeur ajoutée ?

La valeur ajoutée du projet réside dans l'association entre gestion équestre et exploitation expérimentale des données. L'application fournit un environnement unique dans lequel les informations peuvent être consultées et organisées.

7.3 Limites

- IA : performance et données à améliorer.
- Validation : absence de validation clinique.
- Base de données : SQLite adaptée au prototype mais moins adaptée à une forte concurrence.
- Sécurité : renforcement nécessaire avant production.
- Scalabilité : architecture actuelle conçue pour un contexte académique.

7.4 Améliorations

- Construire un dataset réel, plus large et représentatif.
- Calculer precision, recall, F1-score, matrice de confusion et validation croisée.
- Comparer plusieurs modèles sur le même protocole expérimental.
- Ajouter une validation par des professionnels du domaine.
- Renforcer la sécurité et la gestion des rôles.
- Ajouter des tests automatisés.

7.5 Perspectives IA

- Analyse temporelle de l'évolution de la santé.
- Détection d'anomalies.
- Prédiction du risque de blessure.
- Explainable AI avec SHAP.
- Vision par ordinateur pour l'analyse de mouvement et de boiterie.

7.6 Perspectives IoT et Cloud

- Capteurs de fréquence cardiaque et température.
- GPS et suivi d'activité.
- PostgreSQL pour une architecture plus robuste.
- Docker et déploiement cloud.
- Pipeline CI/CD.

7.7 Perspective mobile

Une application mobile complémentaire pourrait permettre l'accès rapide aux fiches chevaux, au calendrier, aux dossiers médicaux et aux alertes.

Conclusion générale

EquiVision AI constitue un prototype académique qui combine gestion équestre, développement web et intelligence artificielle. L'architecture React/TypeScript, FastAPI, SQLite et Random Forest permet de démontrer l'intégration d'un modèle de Machine Learning dans une application concrète.

Le projet atteint ses objectifs principaux de centralisation et d'expérimentation technologique. Toutefois, la performance expérimentale de 50 % du modèle montre que le composant IA doit encore être amélioré avant toute utilisation réelle. L'enrichissement des données, la validation vétérinaire et une évaluation plus complète représentent les prochaines étapes essentielles.

Ce travail a permis de mobiliser les compétences acquises en développement full-stack, gestion de bases de données, conception d'API et Machine Learning, tout en mettant en évidence les contraintes liées au passage d'un prototype académique vers une solution exploitable en conditions réelles.

Bibliographie

1. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
2. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
3. Documentation officielle de FastAPI.
4. Documentation officielle de React.
5. Documentation officielle de SQLAlchemy.
6. Documentation officielle de Pydantic.
7. Documentation officielle de Tailwind CSS.
8. Documentation du projet jsPDF.
9. JSON Web Tokens — documentation et introduction.

Annexes

Cette section peut contenir les diagrammes UML, le schéma de la base de données, les extraits de code significatifs, les exemples de requêtes API, les résultats détaillés des tests et les captures supplémentaires de l'application.